Arbres de suffixes pour recherche de motifs

18 septembre 2023 09:04

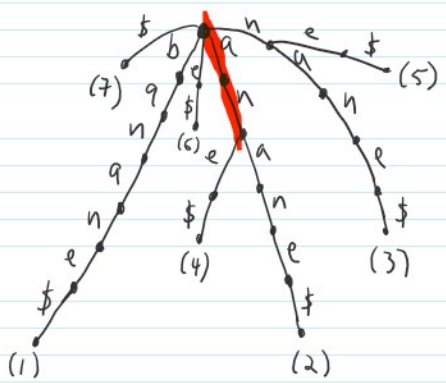
Struct. de données utile pour plusieurs pbs de recherche de séquences

Arbustes de suffixes (trie)

- arbre enraciné
- chaque arête a un symbole
- pour tout noeud, les arêtes enfantes ont un symbole différent
- tout chemin racine-feuille correspond à un suffixe de notre séquence S
  suffixe = $S[i..|S|]$
- on suppose que S termine par un symbole unique $

ex: S = banane $

1 2 3 4 5 6 7

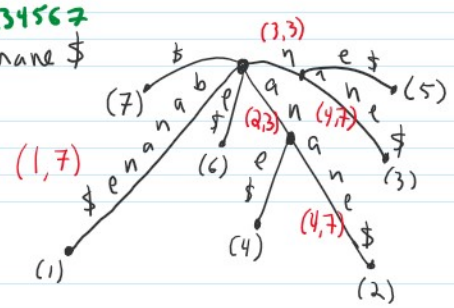


Arbre de suffixes

- arbuste dans lequel on a contracté les noeuds avec un seul enfant
- arêtes étiquetées par des sous-chaînes

1 2 3 4 5 6 7

S = banane $



En réalité, chaque arête est étiquetée par 2 entiers (i,j), telles que $S[i..j]$ est la chaîne à stocker sur l'arête.

- Quel est l'espace en mémoire requis pour stocker un arbre de suffixes? (en fct de $n = |S|$)

→ n feuilles
→ $\leq n$ noeuds internes (exercice)
→ $\leq 2n$ noeuds
→ $\leq 2n$ arêtes

Stocker l'arbre : $O(n)$ espace

(en fait,

Stocker l'arbre : $O(n)$ espace

Espace par feuille : $O(1)$ pour stocker pos. de S (en fait, $\log n$)

Espace par branche : $O(2) = O(1)$ pour paire de pos. de S

$\rightarrow$ Espace total : $O(n)$

Temps de construction :

1) Construire arbuste $O(n^2)$
2) Contracter les noeuds avec 1 enfant $O(n^2)$

Il existe des algos qui construisent les arbres en $O(n)$

- Algo de Ukkonen (1980)
- Algo de Farach-Colton (1990)

Construire arbuste

$S = \overset{i}{\text{banane}}\$$

jjj

Entrée : séq. S

```
r = new Noeud()

pour i = 1 .. |S|
    cur = r
    pour j = i .. |S|
        si cur a un enfant étiqueté S[j]
            cur = l'enfant étiqueté S[j]
        sinon
            v = new Noeud()
            cur.enfants.add(v)
            étiqueter (cur, v) par S[j]
            cur = v
    cur.position = i
```

Recherche exacte de motif

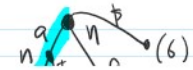
<u>Entrée</u> : séq. P et S

<u>Sortie</u> : liste des occurrences de P dans S

1) Suivre les caractères de P 1 à 1 sur l'arbre A pour S.

$S = \overset{1\,2\,3\,4\,5\,6}{\text{anana}}\$$

A



1) Suivre les caractères de P 1 à 1 sur l'arbre A pour S.

$O(|P|)$

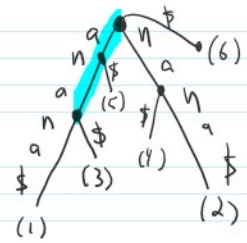
↳ si on n'épuise pas tous les car. de P ⇒ pas d'occurrence

2) soit v le noeud obtenu après le parcours de P.

$O(|S|)$

Pour chaque feuille qui descend de v, lister la position de la feuille comme une occurrence

A

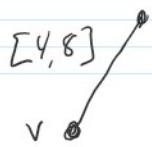


P = ana

↳ en fait, ce temps est prop. au nb d'occurrences de P dans S.

↳ $O(|P| + \#occ)$, où #occ = nb d'occ. de P dans S.

```
struct Noeud
    pos1
    pos2        //positions dans S
    enfants
    pos_suffixe // feuilles
```




```
find (P, A, S)
    cur = A.racine
    ppos = 1
    fini = false, ok = false
    noeud_final = NULL

    tant que ¬fini
        pour v ∈ cur.enfants
            si P[ppos] == S[v.pos1]
                ok = true
                pour apos = v.pos1 .. v.pos2        // suivre l'arête
                    si P[ppos] == S[apos]           // (cur, v)
                        ppos += 1
                    sinon
                        return NULL
                    fin si
                    si ppos > |P|
                        noeud_final = v
                        fini = true
                        break
                    fin si
                fin pour
                cur = v
            fin si
        fin pour
        si ¬ok, return NULL
    fin tant que
    return getFeuilles(noeud_final)

getFeuilles (v)
    si v.enfants.size == 0
        return [v.pos_suffixe]
    retliste = []
    pour w ∈ v.enfants
        retliste.append( getFeuilles(w))
```

```
pour w ∈ v.enfants
    retliste.append( getFeuilles(w))
return retliste
```

Plus longue sous-chaîne répétée

<u>Entrée</u>: séq. S

<u>Sortie</u>: deux sous-chaînes $S[i..i+k]$ et $S[j..j+k]$ égales avec k maximum, $i \neq j$

1 2 3 4 5 6 7 8 9 10 11

ex: mississippi $i=2$ $j=5$ $k=3$

<u>But</u>: trouver un noeud interne dont la <u>profondeur en caractères</u> est maximum.
↳ nb de car. sur le chemin de la racine à ce noeud.

1) Construire l'arbre de suffixes A pour S $O(|S|)$

2) Étiqueter chaque noeud par sa profondeur en caractères $O(|A|)=O(|S|)$

3) Trouver un noeud interne de prof. max $O(|A|)=O(|S|)$

4) Retourner les pos. suffixes de 2 feuilles sous ce noeud $O(|S|)$

Temps: $O(4|S|)=O(|S|)$

Plus longue sous-chaîne commune

<u>Entrée</u>: S et T

<u>Sortie</u>: sous-chaînes $S[i..i+k]$ et $T[j..j+k]$ égales qui maximisent k.

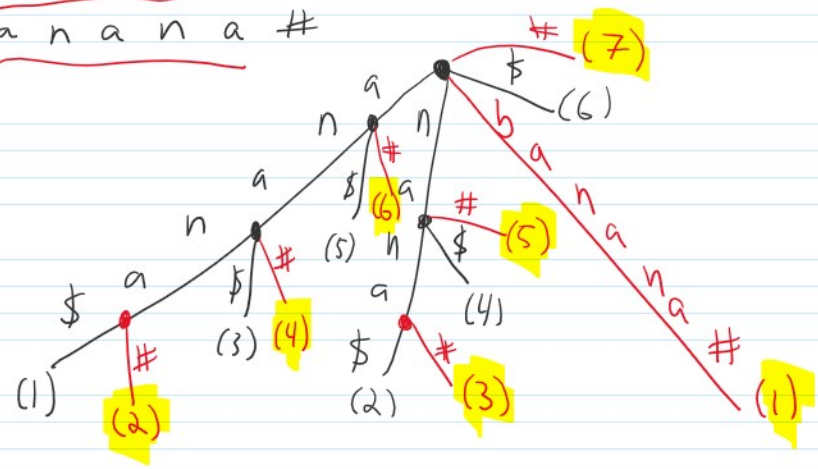
<u>Arbres de suffixes généralisés</u>

Arbre qui contient les suffixes de S et T à la fois.
└ S et T doivent avoir un symbole de fin distinct.

ex: S = a n a n a $

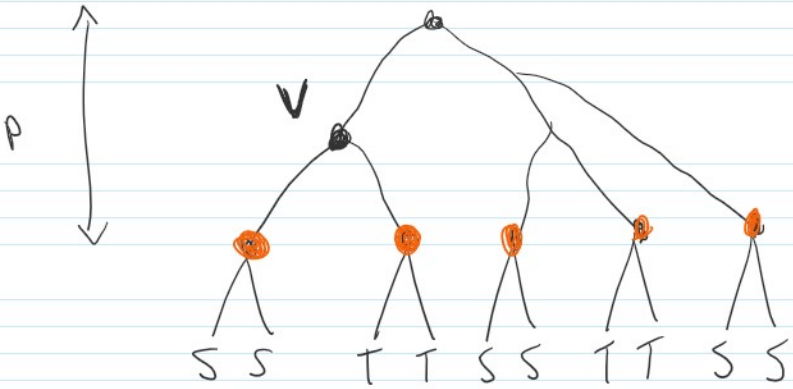
– S et T doivent avoir un symbole de fin distinct.

ex: S = a n a n a $
T = b a n a n a #



1) Construire l'arbre généralisé pour S et T $O(|S|+|T|)$
2) Chercher un noeud v de prof. en caractères max tel que v a un descendant dans S et dans T. $O(|A|)$

[exercice $O(|A|)$]



3) Retourner une feuille de S et de T qui descendent de v. $O(|A|)$

Plus Long Préfixe Commun (PLPC)

Séq. S, pos i, j

PLPC(i, j) = longueur de préfixe commun le + long entre S[i..n] et S[j..n]

Préfixe: sous-chaîne contenant le début

123456
anana$

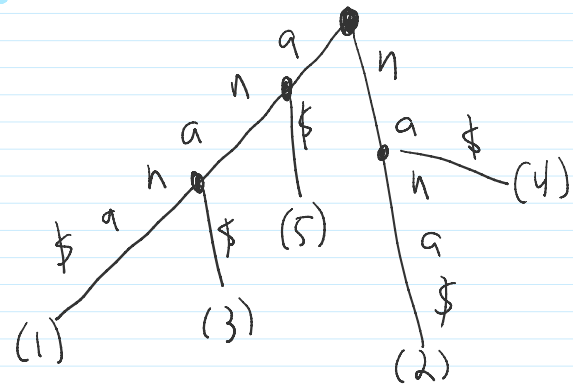
PLPC(2, 4) = 2

PLPC(1, 2) = 3

anana$

$PLPC(2,4) = 2$

$PLPC(1,3) = 3$



Si on a l'arbre de suffixes, $PLPC(i,j)$ est la profondeur en caractères de l'ancêtre commun le + bas des feuilles i et j. (+ bas = + loin de la racine)

$LCA(i,j)$ = dernier ancêtre commun de i et j

lca = least common ancestor

<u>Théorème</u> : il existe un algo qui, étant donné un arbre A, effectue un prétraitement en temps $O(|A|)$ et, ensuite, retourne $LCA(i,j)$ en temps $O(1)$, $\forall i,j$.